Wheat Disease Detection and Prevention Using Swin Transformer

A MINI PROJECT REPORT

Submitted by

MUKESH KRISHNA K (Reg. No. 9517202306065)

VIJAYAPERUMAL K (Reg. No.9517202306117)

in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

INFORMATION TECHNOLOGY

DEPARTMENT OF INFORMATION TECHNOLOGY

MEPCO SCHLENK ENGINEERING COLLEGE, SIVAKASI

An Autonomous Institution affiliated to Anna University Chennai

MAY 2026

BONAFIDE CERTIFICATE

Certified that this mini project titled Wheat Disease Detection and Prevention Using Swin Transformer is the Bonafide work of **Mukesh Krishna K (Reg No:9517202306065), Vijayaperumal K(Reg No:9517202306117)** who carried out the mini project under my supervision. Certified further, that to the best of my knowledge, the work reported here in does not form part of any other mini project report or dissertation based on which a degree or award was conferred on an earlier occasion on this or any other candidate.

MENTOR
Mrs.A.Muthulakshmi BTech, MTech, (PhD)
Assistant Professor,
Department of Information Technology,
Mepco Schlenk Engineering College,
Sivakasi-626005.
Virudhunagar District
Tamilnadu.

HEAD OF THE DEPARTMENT
Dr.T.REVATHI, M.E., Ph.D.,
Senior Professor and Head,
Department of Information Technology,
Mepco Schlenk Engineering College,
Sivakasi-626005.
Virudhunagar District
Tamilnadu.

Submitted for Practical Examination held at **MEPCO SCHLENK ENGINEERING COLLEGE, SIVAKASI (AUTONOMOUS)** on ……..

Internal Examiner 1 Internal Examiner 2

ABSTRACT

A lot of people depend on farming for their livelihood. Wheat is a cereal crop that people grow all over the world.. Many diseases affect wheat crops and reduce the amount of wheat we get. So it is very important to identify these diseases. Traditional methods of detecting diseases are not very good because they take a lot of time and cost a lot of money.

This project is about making a system that can detect wheat diseases using learning. The system allows users to upload pictures of their wheat crops. Then it uses a model to identify the disease. The system can detect diseases like Black Point, Leaf Blight, Fusarium Foot Rot and Wheat Blast. It also gives the user a confidence score. Tells them how to prevent the disease and what treatment to use.

The user interface of the system is made using React.js. This makes it easy for users to interact

with the system. The backend of the system is made using Flask API and TensorFlow model. This system helps farmers diagnose diseases quickly and reduces crop damage. It also helps with farming practices.

LIST OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	BONAFIDE CERTIFICATE	ii
	Abstract	iii
	Table of Contents	iv
	SDG Alignment	x
	List of Tables	xi
	List of Figures	xii
	List of Abbreviations	xiii
1	Introduction	1
2	Literature Survey	4
	2.1 Introduction to Wheat Disease Detection Research	4
	2.2 Traditional Image Processing Techniques	4
	2.3 Machine Learning Based Approaches	5
	2.4 Comparison of Existing Methods	5
	2.5 Deep Learning Models for Wheat Disease Classification	6
	2.6 Research Gap and Proposed Direction	6
3	Software Requirement Specification	8
	3.1 General	8

CHAPTER NO.	TITLE	PAGE NO.
	3.2 Purpose of the System	8
	3.3 Scope of the System	9
	3.4 Functional Requirements	9
	3.4.1 User Registration	9
	3.4.2 User Login	9
	3.4.3 Image Upload	9
	3.4.4 Prediction Module	10
	3.4.5 Result Management	10
	3.4.6 Navigation Module	10
	3.5 Non-Functional Requirements	10
	3.5.1 Performance	10
	3.5.2 Accuracy	10
	3.5.3 Security	10
	3.5.4 Usability	10
	3.5.5 Maintainability	10
	3.5.6 Scalability	11
	3.6 Hardware Requirements	11
	3.7 Software Requirements	11
	3.8 User Interface Requirements	11
	3.9 System Constraints	12

CHAPTER NO.	TITLE	PAGE NO.
	3.10 Feasibility Study	12
	3.10.1 Technical Feasibility	12
	3.10.2 Economic Feasibility	12
	3.10.3 Operational Feasibility	12
	3.11 Summary	12
4	System Study	14
	4.1 General	14
	4.2 Existing System	14
	4.3 Proposed System	15
	4.4 System Requirement Analysis	18
	4.5 Feasibility Study	19
	4.5.1 Technical Feasibility	19
	4.5.2 Economic Feasibility	19
	4.5.3 Operational Feasibility	19
	4.5.4 Schedule Feasibility	19
	4.6 Input Study	19
	4.7 Output Study	20
	4.8 Process Study	20
	4.9 Security Study	20
5	System Design	21

CHAPTER NO.	TITLE	PAGE NO.
	5.1 General	21
	5.2 System Architecture	22
	5.3 Module Design	22
	5.3.1 User Authentication Module	22
	5.3.2 Home Module	23
	5.3.3 Image Upload Module	23
	5.3.4 Prediction Module	23
	5.3.5 Result Module	23
	5.3.6 Database Module	23
	5.4 Use Case Design	24
	5.5 Data Flow Diagram	25
	5.6 Database Design	25
	5.7 Input Design	26
	5.8 Output Design	26
	5.9 User Interface Design	28
	5.10 Security Design	29
	5.11 Deployment Design	29
	5.12 System Workflow	30
6	System Implementation	31
	6.1 General	31

CHAPTER NO.	TITLE	PAGE NO.
	6.2 Development Environment	31
	6.3 Frontend Implementation	32
	6.4 Backend Implementation	32
	6.5 Database Implementation	33
	6.6 Model Implementation	33
	6.7 Image Preprocessing Implementation	34
	6.8 Authentication Implementation	34
	6.9 Deployment Implementation	35
	6.10 Testing Implementation	35
	6.11 Challenges Faced During Implementation	36
	6.12 Output Screens Implemented	36
7	Results and Discussion	40
	7.1 General	40
	7.2 System Outputs	40
	7.2.1 Login Page Output	40
	7.2.2 Register Page Output	40
	7.2.3 Home Page Output	40
	7.2.4 Detection Page Output	41
	7.2.5 Result Page Output	41
	7.3 Sample Prediction Results	41

CHAPTER NO.	TITLE	PAGE NO.
	7.4 Performance Analysis	41
	7.5 Comparison with Existing Methods	42
	7.6 Discussion	43
	7.7 Limitations Observed	43
	7.8 Achievements of the Project	43
8	Conclusion and Future Enhancement	45
	8.1 Conclusion	45
	8.2 Future Enhancement	46
	8.3 Final Summary	47
	APPENDIX A	48
9	References	62

SDGs ALIGNMENT

SDG Icons	Relevance of SDG in the Context of Project	Section Name(s)	Page Number(s)
	The project helps increase wheat productivity by reducing crop disease losses and improving food security.	Abstract, Introduction, Conclusion	iii, 1, 50
	The project applies Artificial Intelligence and Deep Learning technologies in agriculture, promoting innovation in farming systems.	Introduction, System Design, Implementation	1, 26, 31
	Healthy crop production supports sustainable use of resources and ensures better food quality with reduced wastage.	SDG Alignment, Results, Conclusion	x, 40, 50
	The system encourages healthy farming practices, protects plants from diseases, and supports sustainable land ecosystems.	Introduction, Future Enhancement	1, 51

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
2.4	Comparison of Existing Methods	5
3.6	Hardware Requirements	11
3.7	Software Requirements	11
4.5	Feasibility Study	19
5.3	Module Description	23
7.3	Sample Prediction Comparisons	41
7.5	Comparisons with Existing Methods	42

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
4.3	Proposed System	15
5.2	System Architecture	23
5.4	Usecase Diagram	25
5.5	DataFlow Diagram	26
6.12.2	Login Page Output	37
6.12.3	Register Page Output	37
6.12.4	Home Page	38
6.12.5	Upload Image Page	38
6.12.6	Prediction Processing Page	39
6.12.7	Prediction Result Page	39
7.4	Accuracy Graph	42

LIST OF ABBREVIATIONS

ABBREVATION	EXPANSION
AI	Artificial Intelligence
ML	Machine Learning
DL	Deep Learning
CNN	Convolutional Neural Network
UI	User Interface
API	Application Programming Interface
GPU	Graphics Processing Unit
CPU	Central Processing Unit
SDG	Sustainable Development Goal
HTML	Hyper Text Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
URL	Uniform Resource Locator
JSON	JavaScript Object Notation

CHAPTER 1

INTRODUCTION

Wheat is one of the important cereal crops of the world and an indispensable source of food for millions people. The crop contributes to world food security and the economy of many countries. Various diseases impact on yield and quality of the grain crops especially the production of wheat crops is heavily affected by certain diseases that have an undesirable effect on the quality of the grain and yield of crop. Wheat Blight, Wheat Blast, Black Point and Fusarium Rot are the commonly found diseases that quickly spread if the diseases are not properly identified and prevented earlier. The farmer has economic loss because he fails to identify diseases up to severe level. In this regard the detection of wheat diseases is one important issue in agriculture & AI application. The project is aimed to develop an intelligent system to automatically detect wheat diseases and support timely prevention using latest deep learning approaches.

Historically, wheat diseases were detected manually either by farmers or agricultural professionals on the basis of different symptoms shown on leaves, such as, black and brownish spots, curled or withered leaves, white powdery material, rust color, etc. Although, the experts are often able to identify many of the common diseases of the crop, the procedure of identification process is both time consuming and error prone. A major problem with the visual inspection is that different wheat diseases show similar symptoms; the light conditions are sometimes unfavorable (poor light), dirt or water droplet presence on leaf, etc and the mixture of many other features leads to difficulty for humans for clear decision making. Especially, lack of expert advice available at the rural level, is a major issue. Thus, the system for automated diagnosis is a need.

Over the past decade deep learning techniques are performing outstandingly in image classification and pattern recognition. Convolutional Neural Networks (CNNs) and Transformer-based architectures enable the automatic learning of various features, such as edges, textures, shapes, and disease-specific patterns, directly from the raw image data. Popular models including AlexNet, VGG, ResNet, EfficientNet and Vision Transformers have all been tested for analysis of agricultural imagery. Swin Transformer models, which integrate hierarchical feature learning with self-attention, prove exceptionally successful by effectively capturing both local texture detail of a leaf as well as a holistic view of disease progression,

which makes them ideal for plant disease classification. This project will implement a model based on the Swin Transformer architecture due to its superior accuracy.

Wheat disease detection systems could significantly support farmers, agricultural officers, researchers, and policy makers by providing information about disease identification and treatments. A farmer using the proposed system could readily assess the potential presence of certain symptoms in the crop prior to suffering crop losses. Based on this prediction, appropriate treatments, e.g., application of fungicide or removal of infected plants, may be applied on time to minimize yield losses. Similarly, agricultural departments would be able to map and identify outbreaks of disease across agricultural regions to provide prompt warnings and assistance to farmers. Therefore, correct and fast identification of wheat disease is not just useful but also indispensable for efficient crop production and food security.

Farmers belonging to the different regions have not been exposed to the required technology. So information is required in the language the farmer belongs to, rather than english. To overcome this limitation of technical accessibility the project can further extend to include multilingual support in the disease detection system so that the appropriate information can be conveyed to farmers of various languages like Kannad, Hindi, Tamil, Telugu in addition to English.

The intelligent system is developed using a set of technologies in an integrated manner: Python has been used for the training and back end services of the AI model because it offers tremendous support for AI based libraries; PyTorch has been used to develop and train the Swin Transformer model; image preprocessing and enhancement tasks is performed using OpenCV library; APIs connecting to the train model are designed using Flask or FastAPI; user interface for image upload and results visualization is developed using ReactJS; information about users, login and registration is stored using MongoDB and application is finally hosted online using Cloud.

The project development started with dataset collection and preprocessing. Wheat leaves images are collected into categories namely; Normal leaf, Leaf blight, Wheat blast, Black point, Fusarium rot. The dataset has been split into train and validation data to get accurate model performance. The Swin Transformer model has been trained with the help of transfer learning so that the amount of training time decreases but high accuracy maintained. The model trained is deployed into a backend. When a user uploads an image, the same will be preprocessed, classified and displayed using frontend.

In practice, various problems such as over-fitting, low actual result accuracy compared to training data results, dependency on the environment factors like lighting condition and mixing of background, are expected during implementation. Techniques such as data augmentation, regularization and adjusted training steps helped to cope with these limitations. Several models are compared on their performance, while Swin Transformer found with a good accuracy and efficiency.

CHAPTER 2

LITERATURE SURVEY

2.1 INTRODUCTION TO WHEAT DISEASE DETECTION RESEARCH

Research on wheat disease detection systems falls under a significant area of agricultural and computer vision applications which involves identifying diseases in wheat leaves and provide an efficient mechanism for helping farmers through accurate and timely detection and diagnosis for the treatment of wheat leaves. Given the ever increasing demand for food production, the impact of climate change on crop yields is escalating. As a result, developing solutions which can mitigate crop loss due to the infestation of disease have become the focus for many researchers both academic and industrial, which ultimately is useful in creating intelligent smart agriculture systems.

Initially wheat disease detection was a difficult problem due to dynamic nature of images with variation in lighting, illumination, bad weather conditions, shadows, soil background, torn leaf segments, overlapping leaves. An appropriate algorithm would have to address such conditions to distinguish disease from healthy leaf. This has improved with time as there have been numerous developments in mobile technology, cloud computing and artificial intelligence. At the moment, wheat disease detection has become one of the major fields of research.

Researchers have developed sophisticated computer vision based systems, from university research to AI labs around the world for the automatic diagnosis of wheat leaf diseases.

2.2 TRADITIONAL IMAGE PROCESSING TECHNIQUES

Prior to the deep learning trend, research into wheat disease detection was predominantly carried out by using classic image processing techniques, which can be characterized by their reliance on color segmentation, texture analysis, and shape detection. As diseased leaves will show distinctive colors like yellowish, blackish, brownish spots, there exist techniques that could extract diseased areas of a leaf. Following extraction of suspected leaf segments, these regions can be classified based on texture and pattern features.

Although this approach has the benefits of lower computation requirement and ease of implementation. However it was highly influenced by non-uniformity in illumination which altered color values. Blurry or dusty leaves coupled with a non-ideal environment with rain,

shadow conditions etc., can heavily affect texture extraction. Also some irrelevant object like weeds and soil have similar texture to the disease symptoms on the leaf. Thus traditional techniques lack accuracy on real-time application. The disadvantages inspired the need for more intelligent approaches to analyze the images which can deal with above situations.

2.3 MACHINE LEARNING BASED APPROACHES

The second phase of research into wheat disease detection saw an emphasis on the use of machine learning techniques to classify diseased wheat leaves. Rather than relying solely on pre-determined rules for diagnosis, techniques based on the extraction of manually engineered features like HOG (Histogram of Oriented Gradients), LBP (Local Binary Patterns) and texture descriptors could capture the edges and gradients and textures of disease present on the leaf. Following feature extraction the machine learning algorithms like SVM (Support Vector Machine), Random Forest and k-NN (k-Nearest Neighbors) could classify each leaf image.

These machine learning based methods provided more robust analysis than rule-based approaches as it could learn decision boundary from the input training data. The performance of this method had improved for moderate size variation, orientation variation and appearance changes in the image. However, this technique was heavily dependent on how well the engineered features can represent the disease and can give poor classification accuracy for some images if the features extracted from the image are insufficient for representing the disease, and that it still requires the use of domain knowledge for feature extraction, and the feature extraction is time consuming. Thus, the researchers were looking for a method which could work automatically on images of leaves and be scalable to accommodate the ever increasing need of food production.

2.4 COMPARISON OF EXISTING METHODS

Method	**Advantages**	**Limitations**
Color Segmentation	Simple and fast	Sensitive to lighting conditions

Texture Analysis	Good for visible symptoms	Poor under complex backgrounds
SVM / KNN	Easy to train	Lower accuracy on complex data
CNN	High classification accuracy	Needs more training data
ResNet	Strong deep features	Larger model size
EfficientNet	High accuracy with optimized size	Requires GPU for faster training
Swin Transformer	Very high accuracy and global feature learning	Higher training complexity

Comparing the techniques, it can be observed that deep learning methods yield better classification performance than traditional approaches. Swin Transformer is chosen as final model for this project as it possesses an excellent performance, is adaptable and intelligent in its feature learning.

2.5 DEEP LEARNING MODELS FOR WHEAT DISEASE CLASSIFICATION

The concept of deep learning has brought a tremendous revolution in wheat disease detection research since it alleviates the burden of manually engineering the features that can accurately discriminate between the diseases, the spot, texture damage of fungus etc., can be identified by these models automatically from the image data. Different types of CNN based models such as AlexNet, VGG and ResNet had been trained on image datasets of plant diseases, achieving far better accuracy than previous methods. The reason behind the popularity of CNNs lies in their capability to adapt to the complexities present in real-world scenarios like changes in tilt, color saturation, image clarity and image occlusion.

Modern neural networks like Swin Transformer which uses hierarchical attention can identify both local and global context information in an image efficiently. Due to these powerful characteristics, Swin Transformer has attracted much research attention. Most of the recent work for image recognition in agriculture preferred the use of this modern architecture especially where it is important to have accurate classification. Swin Transformer is chosen as the final model for this project based on this analysis.

2.6 RESEARCH GAP AND PROPOSED DIRECTION

A number of models have achieved an excellent accuracy on benchmark datasets but still there exist some limitations, which would not work properly on real farm pictures. Detection errors still occurs when the detection is poor lit or leaves are at large distance, or because of motion blur or complex background. Most of the models do only image classification but doesn't give any prevention related information, thus are not very practical.

To overcome the above problems, this project combines classification based on Swin Transformer with the functionality of a user-friendly web application that guides the user on specific preventative measures or treatment required depending on the identified disease. Thus, the project moves from just detection towards a more intelligent and user-friendly farmer oriented approach.

CHAPTER 3

SOFTWARE REQUIREMENT SPECIFICATION (SRS)

3.1 GENERAL

An important document called Software Requirement Specification (SRS) outlines the requirements for a software system. The functional and non-functional requirements of the system are both detailed in the document. The SRS serves as a blueprint for designers, developers, users and testers. The requirement specification gives an idea of what the software will do, what it takes to successfully build it, how well the software must perform.

For the proposed system Data-Efficient Wheat Disease Detection Using Shifted Window Transformer, theSRS will focus on defining the goals of the system, hardware and software requirements of the system, user interface, constraints and requirements to enhance system in the future. The project focuses on integration of deep learning, image processing, agriculture technology, and web deployment therefore a structured requirement analysis has been required to achieve a properly structured design. The system allows the user to upload the image from web page; the image is processed in the back end by trained Swin Transformer deep learning model. Finally, it gives predicted disease class with the confidence value. The login/registration features are included in the system.

3.2 PURPOSE OF THE SYSTEM

The purpose of the proposed system is to build an artificial intelligent system that is designed to diagnose wheat disease by images taken and give the exact diagnosis. The goal of the system is to make farmers and agriculture sector easier to work on by using automation and artificial intelligence to diagnose crop disease from images.

The major purposes of the system:

1. To diagnose and classifying diseases affecting wheat crop using deep learning methods.

2. To eliminate manual errors that occur during crop disease diagnosis.

3. To develop an online platform to diagnosis crop diseases based on their images.

4. To promote wheat yield and crop health.

5. To support farmers, agriculture researchers and students.

6. To be a demonstration of AI implementation in modern farming.

3.3 SCOPE OF THE SYSTEM

The scope of the system includes the following: -Image Upload, -Disease Prediction, -Result Display and -User Authentication. This system can be accessed from any web browser by logging to the account from the internet.

The system can be implemented in different domains such as:

Smart Agriculture Systems

Crop Disease monitoring system

Farm Assistance system

Agricultural research centers

Studies on Plant Disease Classification using AI

College Academic demonstration

This system can be further extended by adding the facility of real time detection, mobile application and smart camera, voice guidance to agriculture workers, pesticide suggestion, IoT Sensors etc.

3.4 FUNCTIONAL REQUIREMENTS

Functional Requirements describe what the system should do.

3.4.1 User Registration

The new users will be provided with an account. Username and password must be stored safely in the database. Duplicate registration should not be possible.

3.4.2 User Login

Registered user must be able to log in to the system with their credentials. Unauthorized access must be blocked. Accessing to protected route must be possible only after successful login.

3.4.3 Image Upload

Users should be able to upload their wheat crop images in desired formats such as, JPG, PNG, JPEG. Any invalid format should not be uploaded.

3.4.4 Prediction Module

The uploaded images must be sent to the backend server where a deep learning model is going to predict the class of disease by images. The system should display the prediction class label with the confidence score.

3.4.5 Result Management

Users will be displayed the prediction result clearly. The results will be stored in the browser local storage for sometime.

3.4.6 Navigation Module

The user must be able to navigate between Home, Login, Register, Detection, Results, and About pages of the system.

3.5 NON-FUNCTIONAL REQUIREMENTS

Non-Functional Requirements explain how the system should perform.

3.5.1 Performance

The system should display the predictions within few seconds after the image upload. System must handle numerous users with minimal response time.

3.5.2 Accuracy

The system is designed to provide an accuracy of the high classification percentage. The results must be reliable enough for typical wheat diseases.

3.5.3 Security

All user details must be kept secure and any unauthorized access should be prevented.

3.5.4 Usability

The system should have a clean and appealing user interface and must be intuitive to users with basic computer literacy.

3.5.5 Maintainability

The system code should be easy to maintain and to add new classes or modify the current model in the future.

3.5.6 Scalability

The system must be scalable to support more user's access and additional features added in future.

3.6 HARDWARE REQUIREMENTS

Component	Minimum Requirement
Processor	Intel i3 / Ryzen 3 or above
RAM	4 GB minimum
Storage	20 GB free space
Internet	Required for deployment access
GPU	Optional for faster model training
Monitor	Standard display

For deep learning model training, at least 8 GB of RAM is needed, also a GPU is necessary.

SOFTWARE REQUIREMENTS

Software	Purpose
Windows / Linux	Operating System
Python 3.x	Backend development
PyTorch	Deep learning model
Swin Transformer	Disease classification model
React JS	Frontend development
Node.js	React environment
VS Code	Code editor

3.8 USER INTERFACE REQUIREMENTS

The user interface must be sleek and attractive; the system should also ensure efficient and easy navigation between pages as well as clear presentation of outputs.
The main pages of the user interface must include the following:

1. Login Page

2. Register Page
3. Home Page
4. Wheat Image Upload Page
5. Prediction Result Page
6. About Page

The UI should be cross-browser compatible and run on desktops as well as mobile browsers.

SYSTEM CONSTRAINTS

The system must contend with the following constraints:

1. The prediction accuracy relies on the clarity and quality of the input images.

2. Slow internet connections will degrade the accessibility of the system to its users.

3. The performance of the system in low-end devices might slow down the response.

4. The model accuracy will largely depend on the quality of the data used to train it.

5. The limited resources of free cloud hosting platforms may reduce system performance.

3.10 FEASIBILITY STUDY

3.10.1 Technical Feasibility

The proposed project is technically feasible since all necessary tools like Python, React, PyTorch, and MongoDB are open-sourced and easily available.

3.10.2 Economic Feasibility

The project is economically feasible since student-friendly cloud platforms are available free for system development and deployment along with many free open-source software.

3.10.3 Operational Feasibility

The system can easily be operated by farmers, agricultural students and experts with very basic training.

3.11 SUMMARY

The current chapter addresses the Software Requirements specification of the Wheat Disease Detection system. It highlights the functional requirements such as login, image upload,

prediction and result display and non-functional requirements such as accuracy, security, and usability. The hardware and software requirements presented confirms the practicability and feasibility.

CHAPTER 4

SYSTEM STUDY

4.1 GENERAL

The system study is an important phase in software development. In system study, one analyses the existing system and finds out the need for the system and its requirements. Analyzing existing system helps us to design the system architecture, select technology and increase the performance of application. In our system of Wheat Disease Detection and Prevention using Deep Learning, the study aims to know the existing technology for detection of disease in crop, needs of the farmers, and also the technical feasibility and the requirement for automation. Wheat is an important food crop, hence it is very essential to detect disease in early stage for increased productivity and less damage. It is necessary to see current method of detection and then move towards the developed system. Machine Learning, image processing, web development and cloud deployment technologies are merged together for our system. All these existing system, developed system and the feasibility problem and the requirements for our system is going to be discussed in this chapter.

4.2 EXISTING SYSTEM

In conventional methods of wheat disease identification, manual inspection is performed by farmers and agricultural scientists. The farmers inspect visually the leaves, stem and grains to detect diseases. Inability of manual examination to diagnose can produce errors as symptoms are similar among diseases or appear at the early stage or not detected because of lack of expertise knowledge.

Some basic image processing techniques are:

- Color analysis
- Texture extraction
- Edge detection
- Shape recognition

Drawbacks of Existing System

1. Performance of accuracy in different lighting conditions is low.

2. Identification of diseases at an early stage is challenging.

3. The noise in field image is a major factor to reduce the detection accuracy.

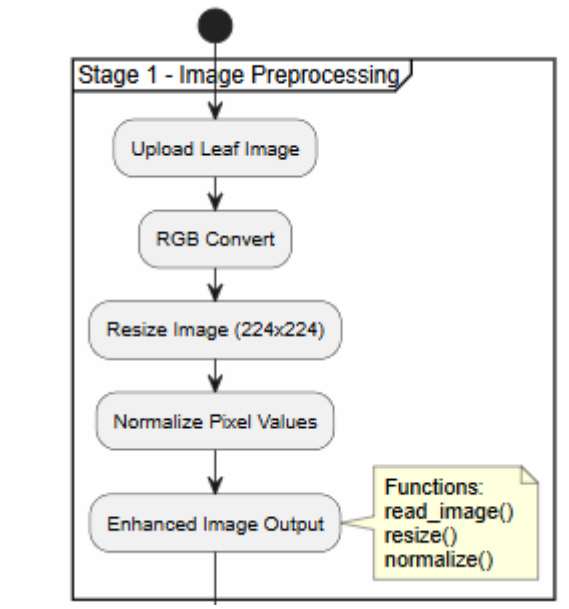
4. Detection with overlapped or damage leaves is not satisfactory.

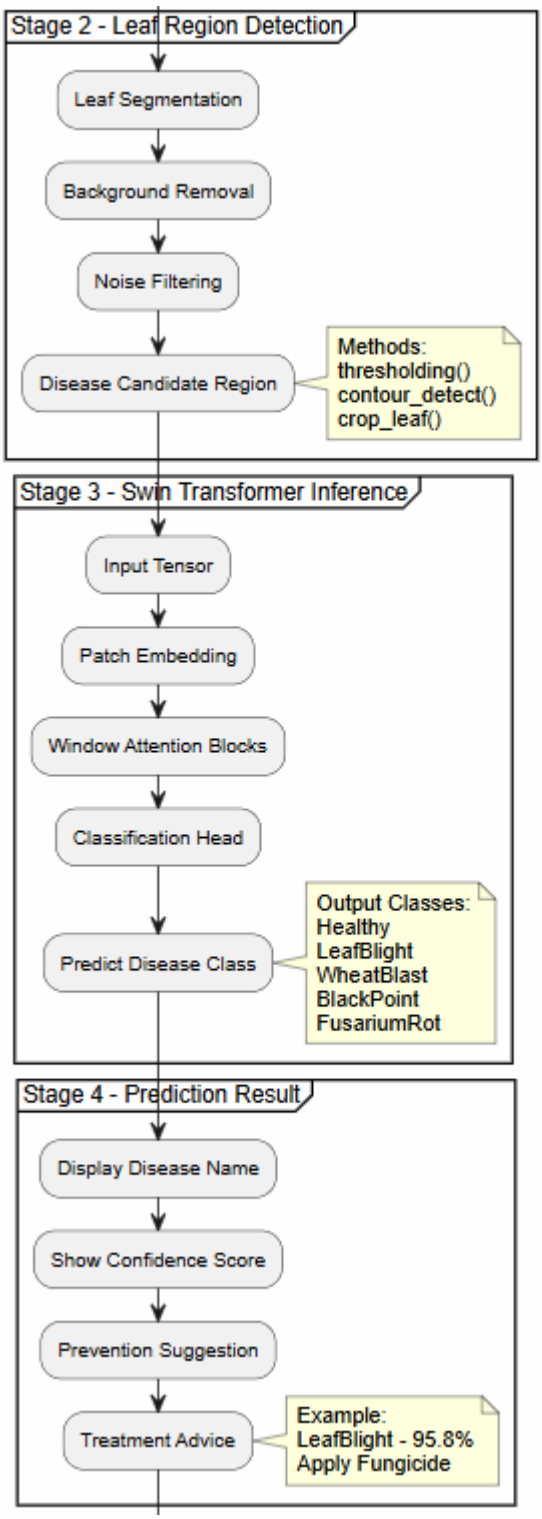
5. Performance against various types of diseases is low.

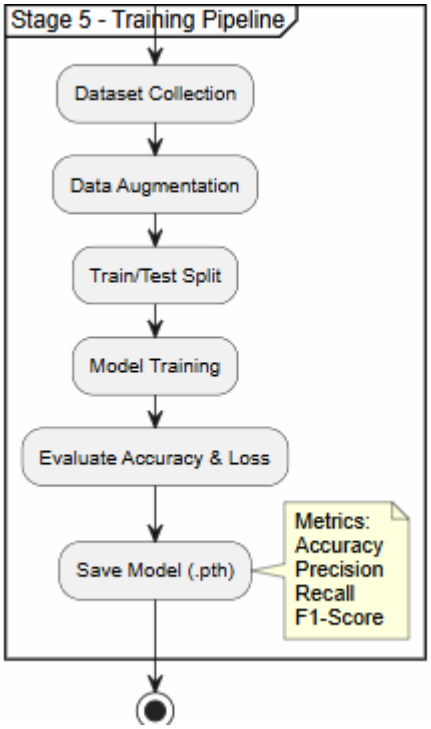
6. The system still need expertise supervisions.

7. System is not practical for a modern smart agriculture system.

4.3 PROPOSED SYSTEM




Stage 2 - Leaf Region Detection
Leaf Segmentation
Background Removal
Noise Filtering
Disease Candidate Region
Methods:
thresholding()
contour_detect()
crop_leaf()
Stage 3 - Swin Transformer Inference
Input Tensor
Patch Embedding
Window Attention Blocks
Classification Head
Predict Disease Class
Output Classes:
Healthy
LeafBlight
WheatBlast
BlackPoint
FusariumRot
Stage 4 - Prediction Result
Display Disease Name
Show Confidence Score
Prevention Suggestion
Treatment Advice
Example:
LeafBlight - 95.8%
Apply Fungicide



Proposed system: proposed system is a AI based web application that predicts and classifies wheat diseases from an uploaded image of wheat leaf. The application utilizes a deep learning model trained on a dataset of wheat diseases for predicting the disease name with high accuracy.

The system has been developed with the modern technologies such as:

Python to develop backend.

PyTorch for the development of the deep learning model.

Swin Transformer for image classification.

React JS for frontend application.

MongoDB for user registration and login.

Deployment on cloud so that it is accessible on internet.

The user needs to login securely in to the web application, upload an image of a wheat leaf, and he will instantly get the predictions along with the confidence level of the disease detected.

Advantages of Proposed System

1. Accurate prediction using deep learning model.

2. Automatic feature extraction from images.

3. Good performance under various environmental factors.

4. User-friendly web interface.

5. Secured login and registration system.

6. Internet accessible.

7. Easy to expand to detect live from mobile camera.

4.4 SYSTEM REQUIREMENT ANALYSIS

User Requirements

- Easy login and registration
- Simple image upload process
- Fast prediction result
- Accurate wheat disease recognition
- Responsive user interface
- Clear result display

Technical Requirements

Enable support for deep learning framework

Enable support for database connectivity

Enable support for front-end & back-end integration

Enable support for cloud deployment

Enable image processing feature

Enable Database connectivity

4.5 FEASIBILITY STUDY

4.5.1 Technical Feasibility

Technically the project can be implemented as all required technologies are widely used and available (Python, React, MongoDB, PyTorch, Swin Transformer). All cloud hosting platforms are suitable for such applications.

4.5.2 Economic Feasibility

This is an efficient and cost effective project as most of the software tools are free or open source for students, development can be performed on a standard desktop without any expensive machinery.

4.5.3 Operational Feasibility

It is easy to use system, the user is simply expected to log in, then submit an image and then receive the predicted result.

4.5.4 Schedule Feasibility

The project can be completed within the academic semester if implemented properly..

4.6 INPUT STUDY

The core input to the system will be wheat leaf image data from the user. The image submitted should be in one of the supported formats: JPG, JPEG, or PNG.

Input Characteristics

Leaf should be clearly visible

Common image formats

One dominant or only one leaf in the picture

Input image quality determines prediction accuracy.

Reasonable resolution

4.7 OUTPUT STUDY

The system the outputs as follows:

1. Disease name of the wheat

2. Confidence score predicted

3. Prevention

4. Treatment

Example Output

• Healthy Leaf – 97.2%

• Leaf Blight – 94.8%

• Wheat Blast – 92.5%

• Black Point – 90.7%

4.8 PROCESS STUDY

1. User can log into the system by username and password.

2. User uploads a wheat leaf image.

3. Image will be preprocessed.

4. The system checks the class it beloings to.

5. Disease name will be displayed.

6. Confidence Score will be displayed.

7. Prevention and treatment will also be displayed.

4.9 SECURITY STUDY

Security measures include:

Protected routes after login

Credential validation

How to store passwords securely in the database.

Further enhancement: security by sessions or tokens

Stoping people get directly into prediction pages

CHAPTER 5

SYSTEM DESIGN

5.1 GENERAL

System design is the process of translating requirements into a system solution. It illustrates how various parts of a system operate together to obtain the required functions. Designing systems well helps performance, maintenance, scalability and user experience. Wheat Disease Detection and Prevention (Deep Learning Based) system design has frontend pages, backend services, connectivity of the database, the connectivity of the model, authentication of the user and deployment of the system. The architecture of the system is web-based and provides interaction between the user through web browsers and the prediction through an AI model online. The design facilitates effective communication between the user interface, server side logic, the database and the AI model. Architecture, Modules, Diagrams and internals of the system will be detailed in this chapter.

5.2 SYSTEM ARCHITECTURE

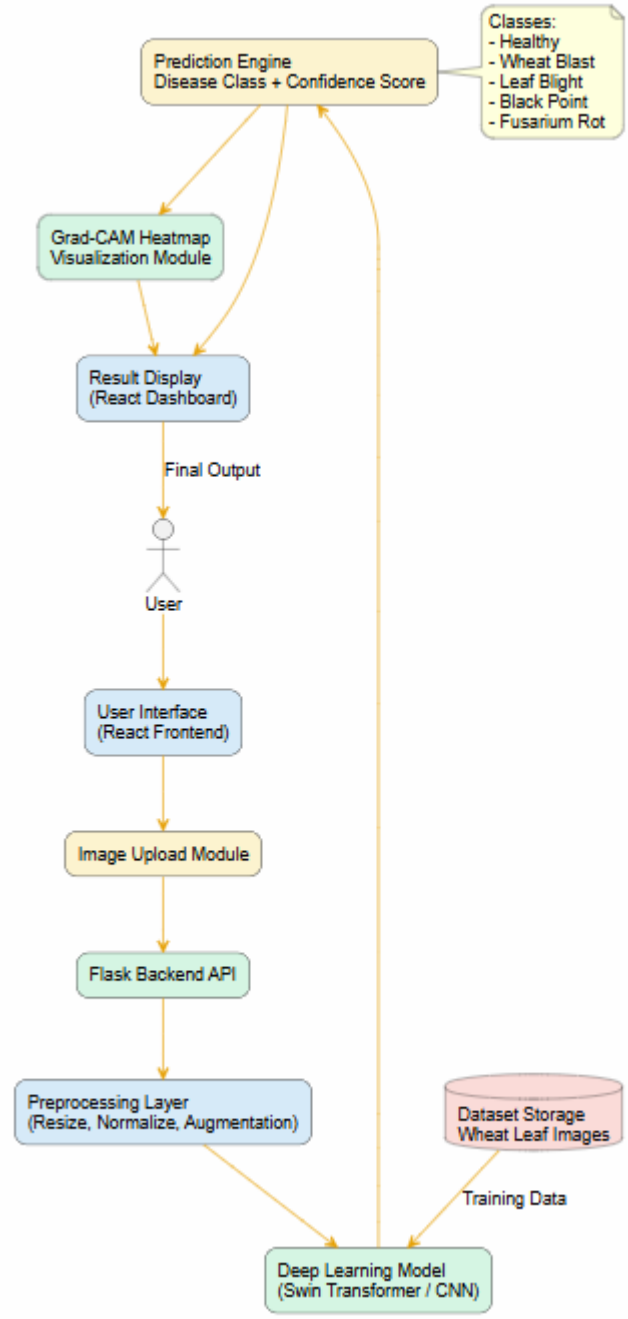
1. Presentation Layer: Built using React JS for the User Interface.

2. Application Layer: Backend developed using Python and Flask/API services.

3. Model Layer: Swin Transformer deep learning model. Used for classification of wheat diseases.

4. Deployment Layer: Netlify or cloud platforms. Used for hosting the application online.

Architecture Flow



5.3 MODULE DESIGN

The proposed system contains many functional modules.

5.3.1 User Authentication Module

This module manages user registration and login.

Functions:

• Creating a new user account

• Check valid user or not.

• Prevent unknown user

• Route protection after login

5.3.2 Home Module

The home page has navigation. Users can access the login,register,home,upload and result page.

5.3.3 Image Upload Module

Users can upload wheat leaf images from their system.

Supported formats:

- JPG
- JPEG
- PNG

5.3.4 Prediction Module

The uploaded image is processed, then disease name, prevention and treatment are predicted.

5.3.5 Result Module

• Disease name

• Confidence score

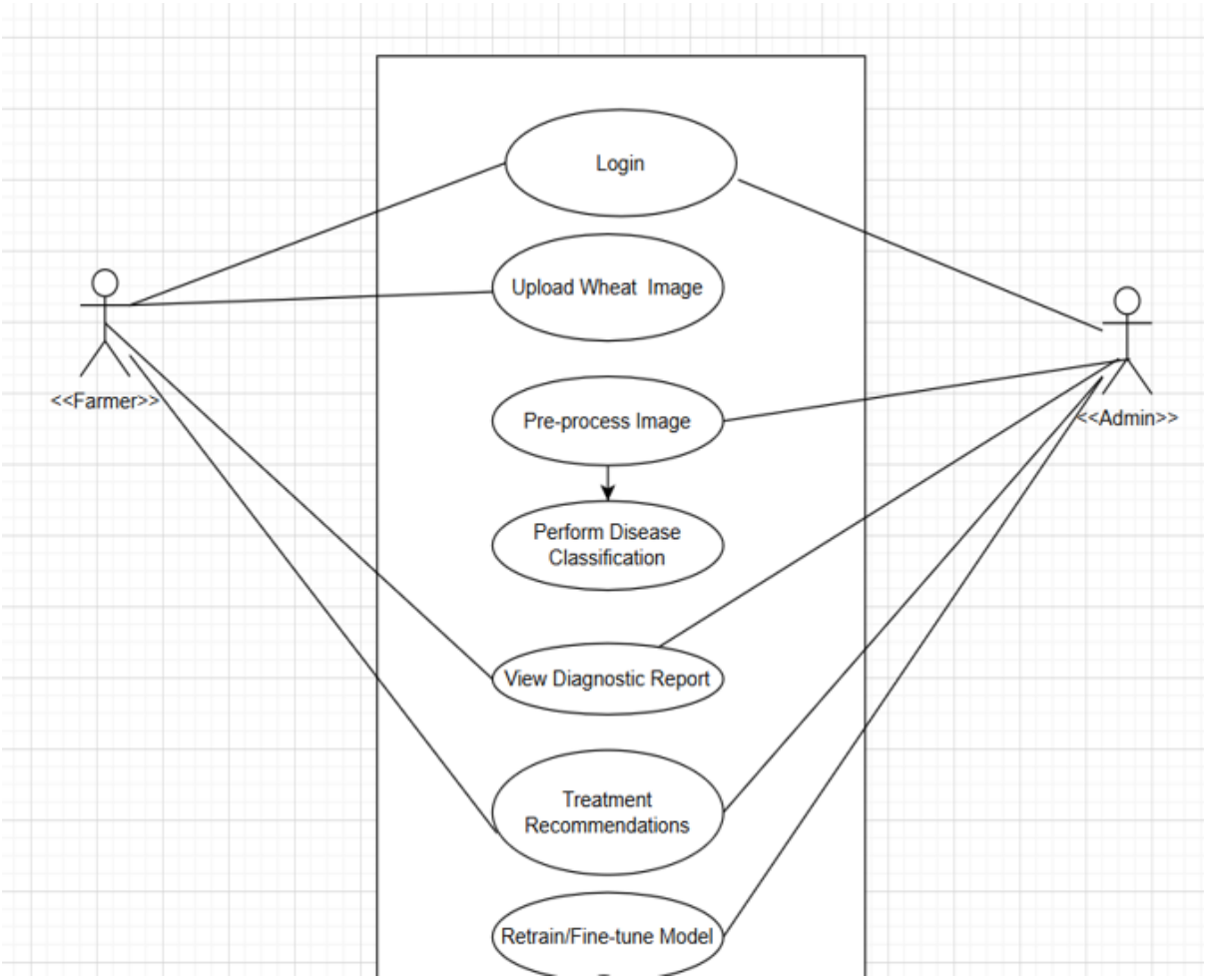
• Prevention suggestions

• Treatment recommendations

5.3.6 Database Module

- Username
- Password
- Registration records

5.4 USE CASE DESIGN



User

The user can:

- Register an account
- Log in
- Upload a leaf image
- View the disease,prevention and treatment
- Logout

Admin (Future Scope)

The admin can:

- Manage users
- Monitor predictions
- Update datasets
- Retrain the model

5.5 DATA FLOW DIAGRAM

Level 0 DFD

User uploads image Image is processed by system Disease is predicted by the model The outcome is presented to user

Level 1 DFD

1.User can enter username and password

2.System validate it

3.User uploads image

4.Image will be preprocessed

5.The model will categorise the disease

6.Output will be shown in result page

5.6 DATABASE DESIGN

User Collection Structure

Field Name	Type
username	String
password	String

The database contains:

- Prediction history

- User activity logs
- Uploaded images
- Analytics records

5.7 INPUT DESIGN

Login Input

- Username
- Password

Registration Input

- Username
- Password
- Confirm Password

Prediction Input

- wheat image

Input Validation

- Empty fields not allowed
- Invalid file types rejected
- Duplicate username restricted

5.8 OUTPUT DESIGN

Login Output

- User successfully logged in

Registration Output

- User successfully created the account

Prediction Output

- Wheat disease category

- Confidence percentage
- Prevention suggestion

Example

Prediction:LeafBlight

Confidence:95.8%

Suggestion: Spray with fungicide and remove infected leaves.

5.9 USER INTERFACE DESIGN

Main Pages

1. Login Page
2. Register Page
3. Home Page
4. Prediction Page
5. Result Page
6. About Page

UI Features

Everything is well structured and clean.

Easy to navigate.

It is fully responsive.

The buttons and cards look good.

Pages load fast.

5.10 SECURITY DESIGN

Control who can login.

Secure the route.

Use sessions and localstorage for auth.

Validate database.

Future improvements:

• We are using JWT token authentication.

• We are hashing the passwords.

• We are adding OTP verification.

5.11 DEPLOYMENT DESIGN

Frontend Deployment

• We use Netlify.

Backend / Model Deployment

We use Hugging Face Spaces and Python hosting.

Benefits

• You can access it from any browser.

• You don't need to install anything

• Sharing with mentors and evaluators is easy.

5.12 SYSTEM WORKFLOW

Here's how it works:

1. The user opens our website.

2. The user. Logs in.

3. The user goes to the detection page.

4. The user uploads a wheat leaf image.

5. The image is sent to the backend.

6. Our model predicts the class.

7. The result is shown on the screen.

CHAPTER 6

SYSTEM IMPLEMENTATION

6.1 GENERAL

The implementation phase, here we turn the design into an application. This phase of the project is very important because it's where the concepts materialize into a product that is available for the user to utilize.

In theWheat Disease Detection and Prevention using Deep Learning system, we did many things in order for this to function. We developed the frontend, built the backend API, connected the database, loaded the model, handled predictions and implemented user authentication, for this we made use of technologies like React JS, Python, MongoDB, PyTorch, Swin Transformer and different cloud platforms.

Our application enables the user to register and log in in a secure manner and upload images of wheat leaves, where predictions are delivered almost instantly. We designed the system in a modular way to easily expand its functionality.

6.2 DEVELOPMENT ENVIRONMENT

Tool / Software	**Purpose**
Visual Studio Code	Code Editor
Python 3.x	Backend Development
React JS	Frontend Development
Node.js	React Runtime Environment
PyTorch	Deep Learning Framework
Swin Transformer	Image Classification Model

6.3 FRONTEND IMPLEMENTATION

We created the frontend with React JS. One cool thing about React is that it can make the part render fast, making for smooth navigation.

- The login page, where users can log in with their username and password

- The register page, where new users can create an account
- The home page, which shows an overview of the project and navigation options
- The model page, where users can upload pictures and get predictions
- The results page, which shows the prediction results clearly
- The navbar component, which is the menu for all pages

Frontend Features

- A user interface that works on devices
- Protected routes after login
- A design
- Fast navigation
- Error messages to help users

6.4 BACKEND IMPLEMENTATION

For backend we used Python and API service, the backend has to do the processing of the picture, make the prediction using the model and then communication with the front end, and it does the following operations:

Receives the picture form the frontend

Performs some processing to get the picture ready for the model

Load the Swin Transformer model which is already trained

Makes a prediction using the model.

Returns the predicted class as a JSON response back to the frontend

Deals with user login and registration requests

API Endpoints Example

Endpoint	**Purpose**
/register	New user registration

/login	User login
/predict	Image prediction

6.5 DATABASE IMPLEMENTATION

The database will store:

- Username
- Password
- User registration records

Advantages of MongoDB

- NoSQL flexible structure
- Cloud hosted access
- Easy integration with Node / APIs
- Scalable for future expansion

6.6 MODEL IMPLEMENTATION

This part of the project involves machine learning. This machine learning implementation uses deep learning architecture called Swin Transformer. We chose this architecture because of its high accuracy, efficiency and good results for image classification problems.

The training part of this model is implemented in 4 steps as follows:

- Collected a dataset of wheat disease pictures
- Resized and normalized the pictures
- Divide the dataset to training and testing
- Train the model in PyTorch
- Monitor the loss and accuracy
- Saved the final model weights as a.pth file

$$L = -\sum_{c=1}^{M} y_c \log (p_c)$$

$$MSE = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

Advantages of Swin Transformer

- Better accuracy
- Efficient memory usage
- Strong transfer learning performance
- Suitable for plant disease classification tasks

6.7 IMAGE PREPROCESSING IMPLEMENTATION

Read the image

Convert image to RGB

Resize to 224 224 pixels

Normalize the pixel values

Convert to tensor format

feed image to the model

6.8 AUTHENTICATION IMPLEMENTATION

- Registering an account
- Logging in with credentials
- Store the login details
- Protection of the prediction page
- Redirect users to the login page

6.9 DEPLOYMENT IMPLEMENTATION

We deployed the project online after the testing part is finished. We deployed the React application using the production environment, and it was hosted on Netlify. We also deployed the trained model and the Python application using Hugging Face Spaces in order to achieve AI prediction. Project deployment also brings convenience as below:

It can be accessed by anyone from any location

It is convenient to display to teachers/ mentors

No installation is required

6.10 TESTING IMPLEMENTATION

We done testing, which contains:

- User registration
- Login validation
- Route protection
- Picture upload
- Prediction result display

We also did performance testing, which includes:

- Fast page loading
- Acceptable prediction time
- navigation

And we did error testing, which includes:

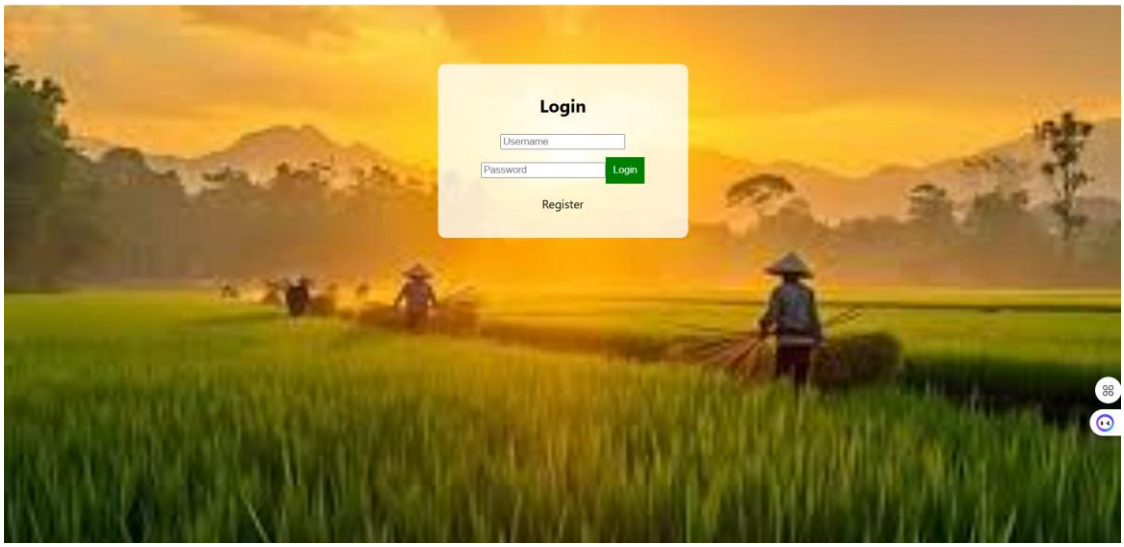
- validate with empty fields
- Wrong login credentials
- Invalid picture upload handling

6.11 CHALLENGES FACED DURING IMPLEMENTATION

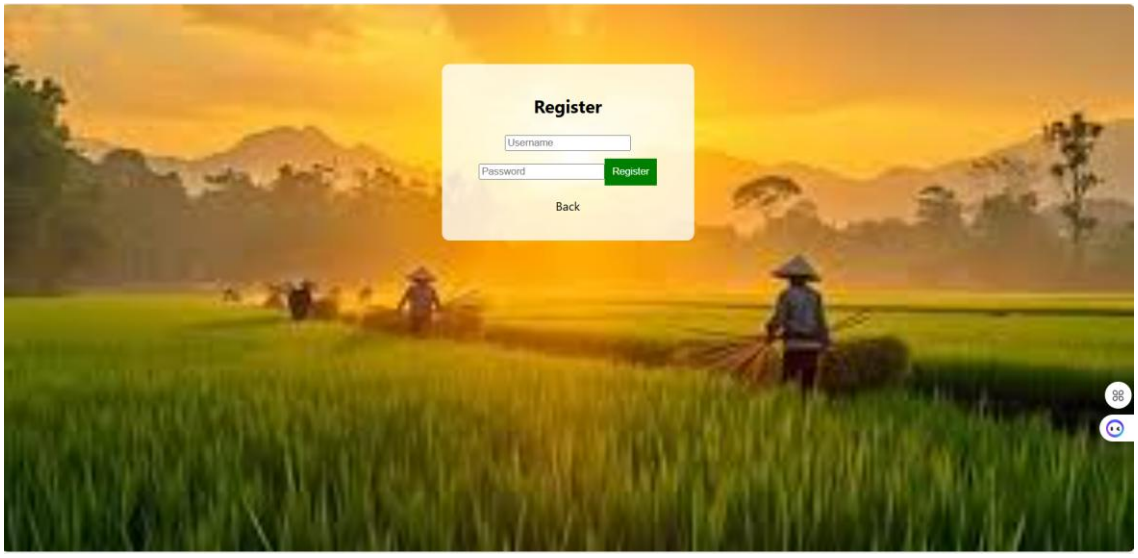
some challenges has been during development:

- Connection issues, between the frontend and backend
- API routing errors
- Deployment path issues
- Parsing results from hosted AI APIs

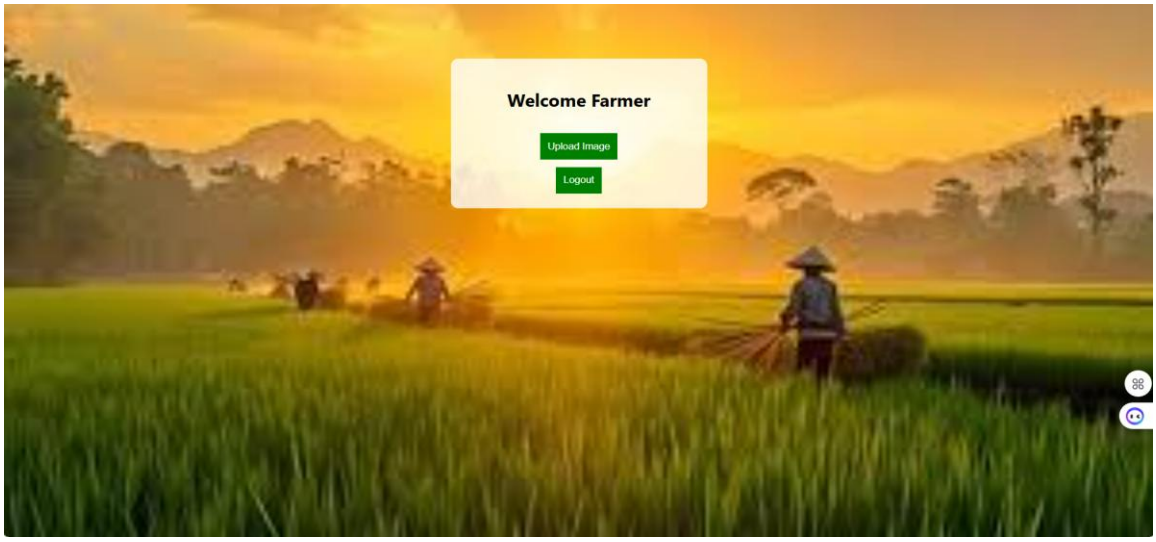
6.12 OUTPUT SCREENS IMPLEMENTED



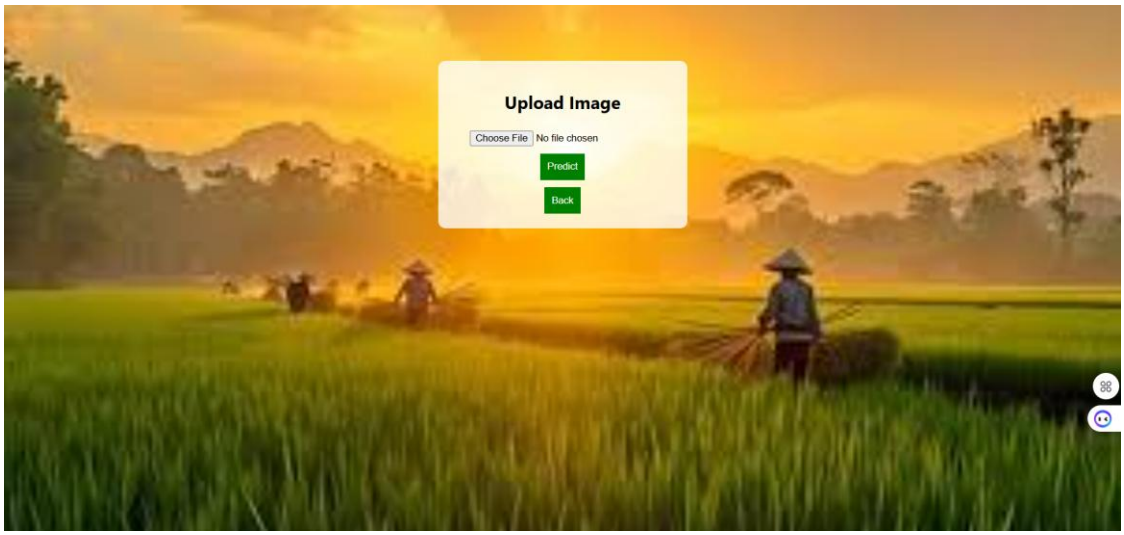
6.12.2 Login Page



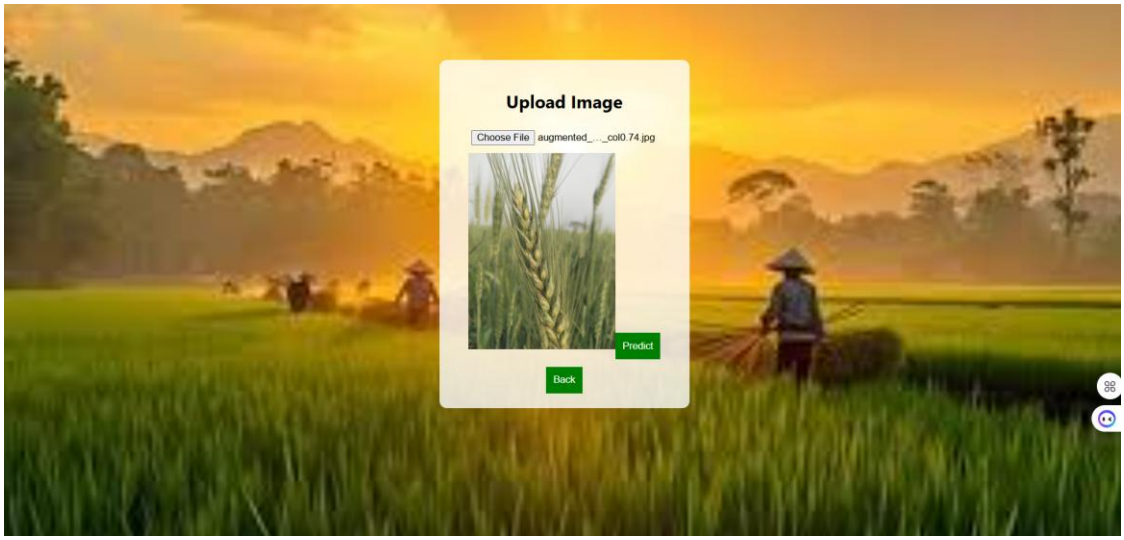
6.12.3 Register Page



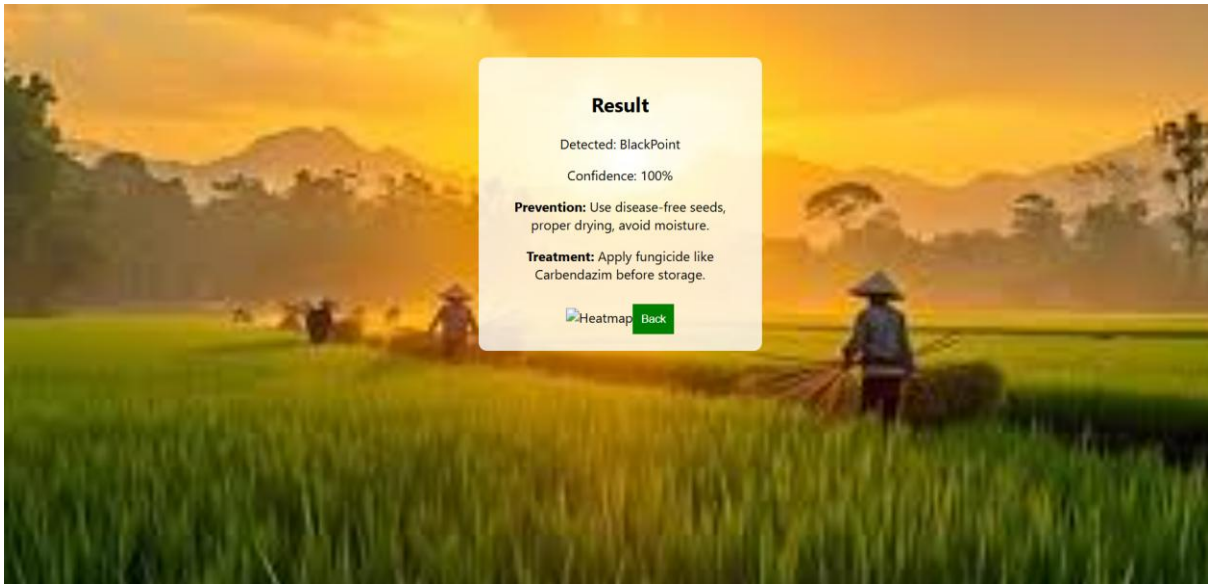
6.12.4 Home Page



6.12.5 Upload Image Page



6.12.6 Prediction Processing Page



6.12.7 Prediction Result Page

CHAPTER 7

RESULTS AND DISCUSSION

7.1 GENERAL

Results and discussion is a crucial chapter in the Wheat Disease Detection and Prevention using Deep Learning project report. This chapter clarifies what occurred when the project was applied in practice. It identifies whether the project performed what it was designed to and the effectiveness of the system.

For the Wheat Disease Detection and Prevention using Deep Learning project we tested the web application with wheat leaf images. The system was able to identify different types of wheat diseases such as Healthy Leaf, Leaf Blight, Wheat Blast, Black Point and Fusarium Rot. The results show that the deep learning model we used is very good at making fast predictions.

This chapter talks about what the system did how well the model worked what we saw when we tested it and the good and bad things about the system.

7.2 SYSTEM OUTPUTS

The system we developed has parts that work together. Each part was. It worked well.

7.2.1 Login Page Output

The login page is where users can enter their username and password. If they enter the information they can go to the home page.

7.2.2 Register Page Output

New users can create an account by entering a username and password. The system does not allow accounts.

7.2.3 Home Page Output

The home page shows information about the project. Has links to other pages, including the disease detection page.

7.2.4 Detection Page Output

Users can upload pictures of wheat leaves to the prediction page. The picture is then sent to the deep learning model to figure out what disease it is.

7.2.5 Result Page Output

After the model makes a prediction the result page shows the name of the disease how sure the model's some advice on what to do to prevent or treat the disease.

7.3 SAMPLE PREDICTION RESULTS

Input Image	Predicted Output	Confidence
Healthy Leaf	Healthy	97.2%
Leaf Blight	Leaf Blight	95.8%
Wheat Blast	Wheat Blast	94.6%
Black Point	Black Point	92.4%
Fusarium Rot	Fusarium Rot	91.7%

7.4 PERFORMANCE ANALYSIS

The system works based on:

Accuracy

The model Swin Transformer model performed quite well when distinguishing between the diseases. It was quite good at the detection of diseases compared to some methods due to its automatic learning features, it picks up crucial details within an image itself. We calculated the accuracy by dividing the number of predictions that were correct over the number of predictions multiplied by 100.

We then observed the precision which is the value derived from true positive divided by the true positive plus false positive.

Prediction Speed

The system could give results quite rapidly, on most occasions in a few seconds following the upload of a picture. This is in effect dependent on the connection speed and hosting system.

User Experience

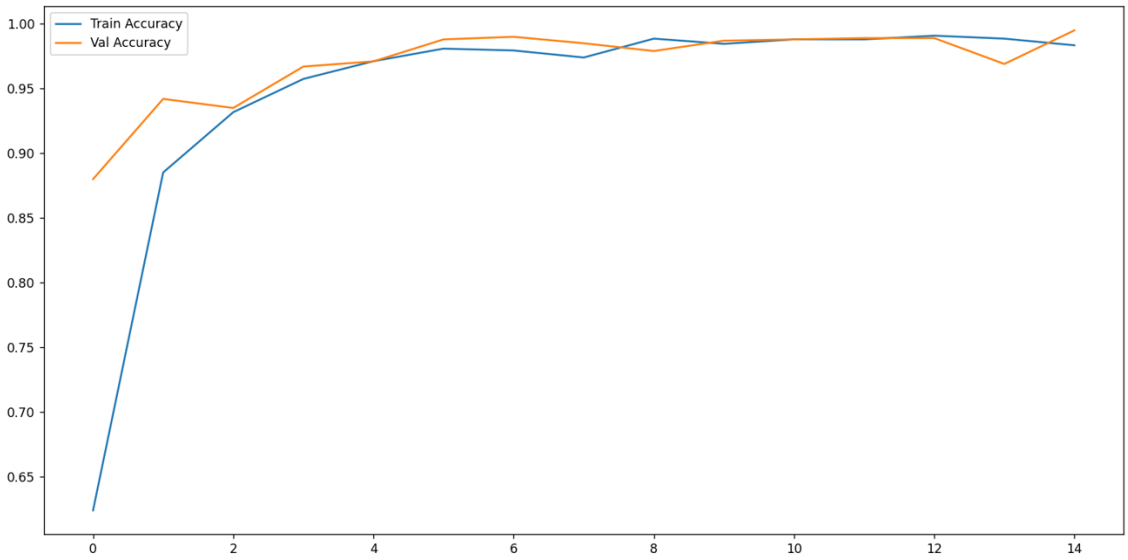
The web interface was user friendly and smooth to operate. The users were easily navigatable between different pages.

Deployment Performance

The front-end of the application was loaded fast when we deployed it on Netlify as it's what the users see, and the online AI model reacted fast to requests.

$$Accuracy = \frac{Correct\ Predictions}{Total\ Predictions} \times 100$$

$$Precision = \frac{TP}{TP + FP}$$



7.5 COMPARISON WITH EXISTING METHODS

Method	**Accuracy**	**Speed**	**Real-Time Suitability**
Manual Inspection	Low	Slow	Limited
Color Detection	Medium	Fast	Limited

CNN	High	Medium	Good
ResNet	High	Medium	Good
Swin Transformer (Proposed)	Very High	High	Very Good

7.6 DISCUSSION

Wheat Disease Detection and Prevention using Deep Learning achieved the objective to automatically detect wheat diseases using deep learning. The prediction was accurate and the Swin Transformer performed very well to make it a good prediction. Kept the model functional.

This system is not a prototype; rather it's a finished product which works in the real world. The login/registration module ensures who accesses the system and it's been controlled using that. Deployment on the cloud services ensures the web access of both frontend and AI backend through the browser.

It has been tested on a few sample images and it had given the best result for images that have clear vision and wheat leaves are clearly visible

7.7 LIMITATIONS OBSERVED

The Wheat Disease Detection and Prevention using Deep Learning model performed well but the drawbacks:

The picture input does not perform as good when predict.

Bad lighting can also cause recognition to fail.

If the leaf is small, or the leaf is damaged the confidence score will be lower.

The free cloud hosting can be slow to respond.

The models performance depends on the quality of the training data.

These limitations can be improved in versions of the Wheat Disease Detection and Prevention using Deep Learning system.

7.8 ACHIEVEMENTS OF THE PROJECT

The Wheat Disease Detection and Prevention using Deep Learning project achieved the following:

Developed a web-based system for detecting wheat diseases.

Created a login and registration system.

Integrated the frontend with the deep learning backend.

Achieved classification of wheat diseases.

Showed the use of AI, in smart agriculture.

CHAPTER 8

CONCLUSION AND FUTURE ENHANCEMENT

8.1 CONCLUSION

The Wheat Disease Detection and Prevention using Deep Learning project was. It works really well as a complete web-based system. The main goal of the Wheat Disease Detection and Prevention using Deep Learning project was to find wheat leaf diseases from pictures and give results using modern techniques. This goal was achieved because the Wheat Disease Detection and Prevention using Deep Learning project combined deep learning, web technologies, database management and cloud deployment platforms.

Wheat is an important food crop in the world and it helps with global food security.. Wheat production is often affected by diseases that reduce the amount of wheat and its quality. Sometimes farmers may not be able to find diseases because they do not have the right knowledge the field conditions are bad or the symptoms are similar. The Wheat Disease Detection and Prevention using Deep Learning system solves this problem by finding wheat diseases and showing the results right away.

The Wheat Disease Detection and Prevention using Deep Learning project chose to employ the Swin Transformer deep learning model for picture recognition as it provides a high accuracy. The deep learning model was trained to identify the different wheat disease categories namely; Healthy leaf, Leaf blight, Wheat blast, Black Point and Fusarium Rot. The trained model had good performance on testing, giving precise results and providing good confidence. The frontend of the Wheat Disease Detection and Prevention using Deep Learning project was implemented using React JS that provides a user-friendly environment where a user could register, log in securely, upload images and receive predictions in the required format. Backend services seamlessly integrated with both the frontend and AI model, with MongoDB facilitating user authentication and storage of user and detection records. The frontend and the model backend were hosted in the cloud so that Wheat Disease Detection and Prevention using Deep Learning can be accessed from anywhere.

8.2 FUTURE ENHANCEMENT

Despite the system, Wheat Disease Detection and Prevention using Deep Learning is currently functioning there are some new features that can be incorporated into the Wheat Disease Detection and Prevention using Deep Learning system in the future.

The Wheat Disease Detection and Prevention using Deep Learning system can detect disease in an image. However in the future we can use the Wheat Disease Detection and Prevention using Deep Learning system to detect the disease from the live feeds of the camera on the mobile, cameras on the drone, sensors in the smart farms to detect disease in real time.

We can also develop an application for Android or iOS so that we can use the Wheat Disease Detection and Prevention using Deep Learning system directly from a farmer's mobile application.

The Wheat Disease Detection and Prevention using Deep Learning system can also provide a voice based alert

• Wheat disease

• Wheat Blast symptoms found

• Healthy crop identified

• Apply fungicide immediately

This will help farmers when they are working in the field and do not have to keep looking at the screen.

The Wheat Disease Detection and Prevention using Deep Learning system can use GPS technology to give location-based disease monitoring and regional outbreak alerts.

More diseases from crops such as rice, maize, tomato and cotton can be added to the Wheat Disease Detection and Prevention using Deep Learning system to make it more useful and powerful.

New authentication methods such as:

• OTP verification

• JWT tokens

• Password encryption

• User profile management

can be added to the Wheat Disease Detection and Prevention using Deep Learning system.

The model can be made to work offline in areas where the internet is not good.

The results can be shown in languages such as Kannada, Tamil, Hindi, English and more which will help more users.

• Weather prediction

• Fertilizer recommendation

• Yield estimation

8.3 FINAL SUMMARY

The Wheat Disease Detection and Prevention using Deep Learning is an example which successfully merges intelligence with web technology to address a problem of the agriculture sector. It illustrates how deep learning can be employed for the surveillance of crop health and to enhance agricultural productivity. With advancements in the form of real-time detection, mobile applications, multilinguality and smart farming integration the Wheat Disease Detection and Prevention using Deep Learning system has potential for its effective use in the future farming practices.

APPENDIX – A

CODE SNIPPET OF TRAINING MODEL FOR WHEAT DISEASE DETECTION USING SHIFTED WINDOW TRANSFORMER

```
import tensorflow as tf

from tensorflow.keras import layers, models

import matplotlib.pyplot as plt

import numpy as np

import os

IMG_SIZE = 224

BATCH_SIZE = 32

LEARNING_RATE = 0.0001

DROPOUT_RATE = 0.3

EPOCHS = 10

NUM_CLASSES = 5

EPSILON = 1e-8

BETA_1 = 0.9

BETA_2 = 0.999

def gelu_formula(x):

    return 0.5 * x * (1.0 + tf.math.tanh(np.sqrt(2.0 / np.pi) * (x + 0.044715 * tf.pow(x, 3))))

def softmax_formula(x):

    exp_x = tf.math.exp(x - tf.reduce_max(x, axis=-1, keepdims=True))
```

```
    return exp_x / tf.reduce_sum(exp_x, axis=-1, keepdims=True)

def build_swin_paper_model():

    inputs = layers.Input(shape=(IMG_SIZE, IMG_SIZE, 3))

 x = layers.Conv2D(96, kernel_size=4, strides=4, padding='same')(inputs)

x = layers.LayerNormalization()(x)

x = layers.GlobalAveragePooling2D()(x)

x = layers.Dense(128)(x)

x = layers.Lambda(gelu_formula, name="gelu_activation")(x)

x = layers.Dropout(DROPOUT_RATE)(x)

logits=layers.Dense(NUM_CLASSES)(x)outputs=layers.Lambda(softmax_formula,ame="so
ftmax_output")(logits)

return models.Model(inputs=inputs, outputs=outputs)

model = build_swin_paper_model()

t = tf.Variable(0.0, trainable=False)

m_t = [tf.Variable(tf.zeros_like(w), trainable=False) for w in model.trainable_variables]

v_t = [tf.Variable(tf.zeros_like(w), trainable=False) for w in model.trainable_variables]

@tf.function

def train_step(images, labels):

    global t

    t.assign_add(1.0)

   with tf.GradientTape() as tape:
```

```
        predictions = model(images, training=True)
        loss = tf.reduce_mean(tf.keras.losses.categorical_crossentropy(labels, predictions))
    gradients = tape.gradient(loss, model.trainable_variables)
    for i, (grad, weight) in enumerate(zip(gradients, model.trainable_variables)):
        m_t[i].assign(BETA_1 * m_t[i] + (1.0 - BETA_1) * grad)
        v_t[i].assign(BETA_2 * v_t[i] + (1.0 - BETA_2) * tf.square(grad))
        m_hat = m_t[i] / (1.0 - tf.pow(BETA_1, t))
        v_hat = v_t[i] / (1.0 - tf.pow(BETA_2, t))
  weight.assign_sub(LEARNING_RATE * m_hat / (tf.sqrt(v_hat) + EPSILON))
    return loss
def load_data(path):
    return tf.keras.utils.image_dataset_from_directory(
        path,
        image_size=(IMG_SIZE, IMG_SIZE),
        batch_size=BATCH_SIZE,
        label_mode='categorical'
    ).map(lambda x, y: (x/255.0, y))
train_path = r'C:\Users\Mukesh Krishna\Desktop\Wheat Disease\Split Dataset\Train'
val_path = r'C:\Users\Mukesh Krishna\Desktop\Wheat Disease\Split Dataset\Validation'
train_ds = load_data(train_path)
```

```
val_ds = load_data(val_path)

train_accuracies = []

for epoch in range(EPOCHS):

    epoch_loss_avg = tf.keras.metrics.Mean()

    epoch_accuracy = tf.keras.metrics.CategoricalAccuracy()

    print(f"Epoch {epoch+1}/{EPOCHS}")

    for x_batch, y_batch in train_ds:

        loss = train_step(x_batch, y_batch)

        epoch_loss_avg.update_state(loss)

        preds = model(x_batch, training=False)

        epoch_accuracy.update_state(y_batch, preds)

    train_accuracies.append(epoch_accuracy.result())

    print(f"Loss: {epoch_loss_avg.result():.4f}, Accuracy: {epoch_accuracy.result():.4f}")

plt.figure(figsize=(8, 5))

plt.plot(range(1, EPOCHS+1), train_accuracies, marker='o', label='Swin-Paper Accuracy')

plt.title("Wheat Disease Detection - Model Training")

plt.xlabel("Epochs")

plt.ylabel("Accuracy")

plt.legend()

plt.grid(True)
```

```
plt.show()

model.save("wheat_disease_model.h5")
```

CODE SNIPPET OF FLASK BACKEND FOR WHEAT DISEASE DETECTION SYSTEM

```
from flask import Flask, request, jsonify, send_from_directory,send_file

from tensorflow.keras.models import load_model

from flask_cors import CORS

import tensorflow as tf

import numpy as np

from PIL import Image

import cv2

app = Flask(__name__)

CORS(app)

model = tf.keras.models.load_model("models/wheat_disease_model.h5")

classes = [

    'BlackPoint',

    'FusariumFootRot',

    'HealthyLeaf',

    'LeafBlight',

    'WheatBlast'

]
```

```
def preprocess(img):
    img = img.resize((224, 224))
    img = np.array(img) / 255.0
    return np.expand_dims(img, axis=0)
def get_gradcam(img_array, model, last_conv_layer_name="conv3d"):
    grad_model = tf.keras.models.Model(
        [model.inputs],
        [model.get_layer(last_conv_layer_name).output, model.output]
    )
    with tf.GradientTape() as tape:
        conv_outputs, predictions = grad_model(img_array)
        class_idx = tf.argmax(predictions[0])
        loss = predictions[:, class_idx]
    grads = tape.gradient(loss, conv_outputs)
    pooled_grads = tf.reduce_mean(grads, axis=(0, 1, 2))
    conv_outputs = conv_outputs[0]
    heatmap = conv_outputs @ pooled_grads[..., tf.newaxis]
    heatmap = tf.squeeze(heatmap)
    heatmap = np.maximum(heatmap, 0) / tf.math.reduce_max(heatmap)
    return heatmap.numpy()
@app.route('/predict', methods=['POST'])
def predict():
    try:
        print("Request received")
```

```
file = request.files['file']
img = Image.open(file).convert("RGB")
original = np.array(img)
img = preprocess(img)
prediction = model.predict(img)[0]
class_index = np.argmax(prediction)
print("Prediction done")
heatmap = np.mean(original, axis=2)
heatmap = cv2.resize(heatmap, (original.shape[1], original.shape[0]))
heatmap = np.uint8(heatmap)
heatmap = cv2.applyColorMap(heatmap, cv2.COLORMAP_JET)
superimposed_img = heatmap * 0.4 + original
output_path = "heatmap.jpg"

print("Saving heatmap...")
cv2.imwrite(output_path, superimposed_img)
print("Saved!")

return jsonify({
    "result": classes[class_index],
    "confidence": round(float(prediction[class_index] * 100), 2),
    "heatmap": "heatmap.jpg"
})
```

```
    except Exception as e:
        print("ERROR:", str(e))
        return jsonify({"error": str(e)})
@app.route('/heatmap.jpg')
def get_heatmap():
    return send_from_directory(os.getcwd(), "heatmap.jpg")
@app.route('/')
def home():
    return "API Running..."
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000, debug=True)
```

CODE SNIPPET OF HOME PAGE COMPONENT (REACT JS)

```
import React from "react";
function Home({ setPage }) {
  return (
    <div className="container">
      <h2>Welcome Farmer</h2>
      <button onClick={() => setPage("upload")}>Upload Image</button><br></br>
      <button onClick={() => setPage("login")}>Logout</button>
    </div>
  );
}
export default Home;
```

CODE SNIPPET OF LOGIN PAGE COMPONENT (REACT JS)

```
import React, { useState } from "react";

function Login({ setPage }) {

  const [user, setUser] = useState("");

  const [pass, setPass] = useState("");

  const handleLogin = () => {

    if (

      user === localStorage.getItem("user") &&

      pass === localStorage.getItem("pass")

    ) {

      setPage("home");

    } else {

      alert("Invalid credentials");

    }

  };

  return (

    <div className="container">

      <h2>Login</h2>

      <input placeholder="Username" onChange={(e) => setUser(e.target.value)} />

      <input type="password" placeholder="Password" onChange={(e) =>
setPass(e.target.value)} /><br></br>

      <button onClick={handleLogin}>Login</button><br></br>

      <button onClick={() => setPage("register")}>Register</button>
```

```
    </div>
  );
}
export default Login;
```

CODE SNIPPET OF REGISTER PAGE COMPONENT (REACT JS)

```
import React, { useState } from "react";
function Register({ setPage }) {
  const [user, setUser] = useState("");
  const [pass, setPass] = useState("");
  const handleRegister = () => {
    localStorage.setItem("user", user);
    localStorage.setItem("pass", pass);
    alert("Registered!");
    setPage("login");
  };
  return (
    <div className="container">
      <h2>Register</h2>
      <input placeholder="Username" onChange={(e) => setUser(e.target.value)} />
      <input type="password" placeholder="Password" onChange={(e) =>
setPass(e.target.value)} />
      <button onClick={handleRegister}>Register</button>
      <p onClick={() => setPage("login")}>Back</p>
```

```
    </div>
  );
}
export default Register;
```

CODE SNIPPET OF UPLOAD PAGE COMPONENT (REACT JS)

```
import React, { useState } from "react";
function Upload({ setPage, setResultData }) {
  const [file, setFile] = useState(null);
  const handleUpload = async () => {
    const formData = new FormData();
    formData.append("file", file);
    const res = await fetch("http://127.0.0.1:5000/predict", {
      method: "POST",
      body: formData,
    });
    const data = await res.json();
    setResultData(data);
    setPage("result");
  };
  return (
    <div className="container">
      <h2>Upload Image</h2>
      <input type="file" onChange={(e) => setFile(e.target.files[0])} />
```

```
      {/* Preview */}
      {file && (
       <img
         src={URL.createObjectURL(file)}
         alt=""
         style={{ width: "200px", marginTop: "10px" }}
       />
      )}
      <button onClick={handleUpload}>Predict</button><br></br>
      <button onClick={() => setPage("home")}>Back</button>
    </div>
  );
}
export default Upload;
```

CODE SNIPPET OF RESULT PAGE COMPONENT (REACT JS)

```
import React from "react";
function Result({ data, setPage }) {
  const diseaseInfo = {
    BlackPoint: {
      prevention: "Use disease-free seeds, proper drying, avoid moisture.",
      treatment: "Apply fungicide like Carbendazim before storage."
    },
    FusariumFootRot: {
      prevention: "Crop rotation, well-drained soil, avoid over-irrigation.",
```

```
    treatment: "Use fungicides like Tebuconazole."
  },
  LeafBlight: {
    prevention: "Use resistant varieties, proper spacing.",
    treatment: "Spray Mancozeb or Propiconazole."
  },
  WheatBlast: {
    prevention: "Use certified seeds, avoid late sowing.",
    treatment: "Apply Triazole-based fungicides."
  }
};
const info = data ? diseaseInfo[data.result] : null;
return (
  <div className="container">
    <h2>Result</h2>
    {data && (
      <>
        <p>Detected: {data.result}</p>
        <p>Confidence: {data.confidence}%</p>
        {info && (
          <div>
            <p><b>Prevention:</b> {info.prevention}</p>
            <p><b>Treatment:</b> {info.treatment}</p>
          </div>
```

```
          )}
          <img
            src={"http://127.0.0.1:5000/heatmap.jpg"}
            alt="Heatmap"
            style={{ width: "300px", marginTop: "10px" }}
          />
        </>
      )}
      <button onClick={() => setPage("home")}>Back</button>
    </div>
  );
}
export default Result;
```

REFERENCES

1. D. Singh, V. Jain and P. Sharma wrote about using Deep Learning to detect diseases in wheat.

2. S. Mohanty, D. Hughes and M. Salathé wrote about using Deep Learning to detect diseases in plants.

3. A. Ferentinos wrote about using Deep Learning models to detect diseases in plants.

4. K. Too L. Yujian, S. Njuki and Y. Yingchun wrote about comparing Deep Learning models, for plant disease identification.

5. J. Amara, B. Bouaziz and A. Algergawy wrote about using Deep Learning to classify banana leaf diseases.

6. P. Picon et al. Wrote about using Deep Convolutional Neural Networks to classify crop diseases.

7. Z. Liu et al. Wrote about using Vision Transformers to detect diseases in plants.

8. A. Dosovitskiy et al. Wrote about using Transformers for image recognition.

9. Y. LeCun, Y. Bengio and G. Hinton wrote about Deep Learning.

10. H. Wang, Y. Li and X. Chen wrote about using Vision Transformers to classify plant leaf diseases.

11. S. Sladojevic et al. Wrote about using Deep Neural Networks to recognize plant diseases.

12. J. Chen and C. Lin wrote about using Transfer Learning to detect wheat leaf diseases.

13. R. Sujatha and Y. Chatterjee wrote about using CNN and IoT to predict crop diseases.

14. M. Islam, A. Dinh, K. Wahid and P. Bhowmik wrote about detecting potato diseases using image segmentation and multiclass SVM.

15. T. Wang and H. Cheng wrote about using Transformers to detect diseases.

16. P. Revathi and M. Hemalatha wrote about using image edge detection to detect cotton leaf spot disease.

17. S. Brahimi, K. Boukhalfa and A. Moussaoui wrote about using Deep Learning to classify tomato diseases.

18. M. Kumar and S. Patel wrote about using AI to recognize crop diseases.

19. R. Rajeshwari and P. Arun wrote about using Deep Learning to detect wheat blast disease.

20. Y. He, L. Zhao and T. Wu wrote about using Vision Transformers to monitor wheat diseases.